

TASK 5: ENCAPSULATION & ABSTRACTION

Title:-

Practical Implementation of Encapsulation and Abstraction Using Python.

1. Introduction:-

Object-Oriented Programming (OOP) is a programming approach that organizes programs using classes and objects. It helps in developing programs that are modular, reusable, secure, and easy to maintain.

Two important concepts of OOP are Encapsulation and Abstraction.

Encapsulation is the process of combining data and methods within a class and controlling direct access to the data. In Python, private attributes can be created using double underscores (__). The data can then be accessed or modified through appropriate methods.

Abstraction is the process of hiding unnecessary implementation details and showing only the essential features to the user. In Python, abstraction can be implemented using the abc module and abstract methods.

In this task, a Bank Account Management System is implemented to demonstrate both encapsulation and abstraction in a practical way.

2. Aim:-

To develop a Python program that demonstrates the concepts of Encapsulation and Abstraction using classes, private attributes, methods, an abstract class, and an abstract method.

3. Objectives:-

The main objectives of this program are:

1. To understand the concept of encapsulation in Python.
2. To use private attributes to protect object data.
3. To provide controlled access to private data using methods.
4. To understand abstraction using an abstract class.
5. To define and implement an abstract method.
6. To create objects and demonstrate the final implementation.
7. To develop a practical application using OOP concepts.

4. Concepts Used:-

4.1 Encapsulation

Encapsulation means wrapping data and methods together inside a class and restricting direct access to the internal data.

In this program, the bank balance is declared as:

```
self.__balance
```

The double underscore makes balance a private attribute. It cannot be accessed directly from outside the class.

Methods such as deposit(), withdraw(), and get_balance() are used to access or modify the balance in a controlled manner.

4.2 Abstraction:-

Abstraction means hiding the internal implementation details and exposing only the necessary functionality.

Python provides the abc module for implementing abstraction.

In this program:

```
class Account(ABC):
```

is an abstract class, and:

```
@abstractmethod  
def display_account(self):
```

is an abstract method.

The child class SavingsAccount provides the actual implementation of the abstract method.

5. Requirements:-

Hardware Requirements

- Computer/Laptop
- Minimum 4 GB RAM
- Keyboard and Monitor

Software Requirements

- Python 3.x
- VS Code / IDLE / PyCharm / Spyder
- Windows/Linux operating system

Python Module Used

- from abc import ABC, abstractmethod

6. Program:-

```
1  from abc import ABC, abstractmethod
2
3
4  # Abstract class
5  class Account(ABC):
6
7      def __init__(self, account_number,
holder_name):
8          self.account_number =
account_number
9          self.holder_name = holder_name
10
11      @abstractmethod
12      def display_account(self):
13          pass
14
15
16  # Child class
17  class SavingsAccount(Account):
18
19      def __init__(self, account_number,
holder_name, balance):
20          super().__init__(account_number,
holder_name)
21
22          # Private attribute for encapsulation
23          self.__balance = balance
24
25      # Method to deposit money
```

```
28         self.__balance += amount
29         print("Amount deposited
successfully:", amount)
30     else:
31         print("Invalid deposit amount.")
32
33     # Method to withdraw money
34     def withdraw(self, amount):
35         if amount > 0 and amount <= self.
__balance:
36             self.__balance -= amount
37             print("Amount withdrawn
successfully:", amount)
38         else:
39             print("Insufficient balance or
invalid amount.")
40
41     # Method to access private balance
42     def get_balance(self):
43         return self.__balance
44
45     # Implementation of abstract method
46     def display_account(self):
47         print("\n----- Account Details -----")
48         print("Account Number :", self.
account_number)
49         print("Account Holder :", self.
holder_name)
50         print("Current Balance:", self.
```

```
49     print("Account Holder :", self.  
holder_name)  
50     print("Current Balance:", self.  
__balance)  
51  
52  
53     # Creating an object  
54     account = SavingsAccount("SB1001",  
"Amrutha", 10000)  
55  
56     # Display initial account details  
57     account.display_account()  
58  
59     # Deposit money  
60     account.deposit(3000)  
61  
62     # Withdraw money  
63     account.withdraw(2000)  
64  
65     # Display updated details  
66     account.display_account()  
67  
68     # Accessing balance through a method  
69     print("\nBalance accessed using method:",  
account.get_balance())
```

Out put:-

```
----- Account Details -----  
Account Number : SB1001  
Account Holder : Amrutha  
Current Balance: 10000  
Amount deposited successfully: 3000  
Amount withdrawn successfully: 2000  
  
----- Account Details -----  
Account Number : SB1001  
Account Holder : Amrutha  
Current Balance: 11000  
  
Balance accessed using method: 11000  
  
[Program finished]
```

8. Explanation of the Program:-

Step 1:

Importing ABC

```
from abc import ABC,  
abstractmethod
```

The ABC and abstractmethod are imported from Python's abc module to implement abstraction.

Step 2:

Creating Abstract Class

```
class Account(ABC):
```

Account is created as an abstract class. It contains common information related to a bank account.

Step 3:

Creating Abstract Method

```
@abstractmethod
def display_account(self):
    pass
```

display_account() is declared as an abstract method. The child class must implement this method.

Step 4:

Creating Child Class

```
class SavingsAccount(Account):
```

SavingsAccount inherits from the abstract Account class.

Step 5:

Encapsulation

```
self.__balance = balance
```

The balance is declared as a private attribute using __.

Therefore, the balance is protected from direct access outside the class.

Step 6:

Controlled Access

The following methods provide controlled access to the private balance:

```
deposit()
withdraw()
get_balance()
```

The deposit() method increases the balance, while withdraw() decreases the balance after checking the available balance.

Step 7:

Implementing Abstraction

The abstract method:

```
display_account()
```

is implemented inside the SavingsAccount class.

Step 8:

Creating Object

```
account = SavingsAccount("SB1001", "Amrutha", 10000)
```

An object of SavingsAccount is created with account number, account holder name, and initial balance.

9. How Encapsulation is Used:-

Encapsulation is implemented using the private attribute:

```
self.__balance
```

The balance cannot be directly accessed or modified from outside the class. Instead, methods are provided:

- deposit() – adds money to the account.
- withdraw() – withdraws money after validation.
- get_balance() – returns the current balance.
-

This provides controlled access to the object's data and helps protect the data from unwanted modification.

10. How Abstraction is Used:-

Abstraction is implemented using the ABC class and @abstractmethod.

The class:

Account

acts as an abstract class.

The method:

```
display_account()
```

is declared as an abstract method.

The SavingsAccount class implements this method and provides the actual functionality.

Thus, the user can use the account features without needing to know the internal implementation details.

11. Advantages:-

Advantages of Encapsulation

- Protects the data from direct access.
- Provides controlled access to data.
- Improves data security.
- Makes the program easier to maintain.
- Prevents unwanted modification of important data.

Advantages of Abstraction

- Hides unnecessary implementation details.
- Makes programs easier to understand.
- Reduces complexity.
- Improves code organization.
- Supports reusable and maintainable code.

12. Real-World Application:-

Encapsulation and abstraction are widely used in real-world software applications.

Examples include:

- Banking systems
- ATM applications
- Student management systems
- Hospital management systems
- E-commerce applications
- Payment applications
- Employee management systems
-

For example, in a banking application, a customer's account balance should not be directly changed by anyone. Instead, operations such as deposit and withdrawal are performed through controlled methods.

13. Result:-

The Python program was successfully implemented to demonstrate Encapsulation and Abstraction.

The program uses a private balance attribute to provide controlled access to account data. It also uses an abstract class and abstract method to hide implementation details and provide a common structure for account classes.

The program successfully performs account operations such as displaying account details, depositing money, withdrawing money, and accessing the balance through a method.

14. Conclusion:-

This task successfully demonstrates the practical implementation of Encapsulation and Abstraction in Python using Object-Oriented Programming.

Encapsulation protects the internal data of an object by restricting direct access and providing controlled methods for data manipulation. Abstraction hides unnecessary implementation details and allows the user to interact only with the essential features of the application.

Through the Bank Account Management System, both concepts were implemented successfully using an abstract class, abstract method, private attribute, constructors, and object methods.

Therefore, the program demonstrates how Encapsulation improves data protection and Abstraction reduces complexity, making Python applications more secure, organized, reusable, and maintainable.